

Taller práctico: del gráfico a la lógica

Ordenamiento burbuja

Estructura de datos

Hecho por: Ashley Dayanne Rodríguez Acosta

FASE I: Exploración y Visualización Dinámica

1. Preparación de la Visualización

Recurso	Detalle
Código/Pseudocódigo	Código de ordenamiento burbuja en lenguaje de programación Python
Herramienta	Visualizer
Datos	Entrada: [5, 2, 9, 1, 5, 6]

2. Observación Activa

1. ¿Cuál es el patrón principal de movimiento o de acción? *El algoritmo compara elementos vecinos y los intercambia si el de la izquierda es mayor. En cada pasada, el elemento más grande “sube” al final de la lista, como una burbuja.*
2. ¿Cómo “detecta” visualmente el algoritmo que ha terminado su trabajo? *En Algorithm Visualizer, los elementos dejan de cambiar de posición y se muestran todos en color verde, indicando que la lista está completamente ordenada.*
3. Si el algoritmo tiene una fase de “división” y otra de “unión”, ¿cómo se distingue el cambio? *Bubble Sort no se divide en fases. Solo tiene un proceso repetitivo de comparaciones e intercambios. Sin embargo, visualmente se nota el cambio de una pasada a otra, ya que los elementos finales quedan fijos al completarse cada iteración externa.*

FASE II: Prueba de Escritorio (Trazado de la Lógica)

1. Caso de Prueba y Planificación

Elemento	Descripción de la tarea
Caso de prueba	lista = [4, 2, 3, 1]
Tabla de trazado	Variables Críticas i, j, lista[j], lista[j+1]
	Condición lista[j] > lista[j+1]

2. Ejecución Manual (Simulación paso a paso)

Paso	i	j	lista[j]	lista[j+1]	Condición(lista[j]>lista[j+1])	Resultado
1	0	0	4	2	4 > 2 → True	Se intercambian → [2, 4, 3, 1]
2	0	1	4	3	4 > 3 → True	Se intercambian → [2, 3, 4, 1]
3	0	2	4	1	4 > 1 → True	Se intercambian → [2, 3, 1, 4]
4	1	0	2	3	2 > 3 → False	Sin cambio

FASE III: Validación y Reflexión

1. Validación de la Lógica

Se ejecutó el mismo caso en Algorithm Visualizer, observando que los intercambios se producen exactamente en los mismos pares y en el mismo orden.

✅ La tabla de trazado manual coincide con la ejecución visual.

2. Reporte de Conclusión

- **Conexión Visual-Lógica:**

Cuando en la visualización se ve que dos barras cambian de color y se cruzan, eso representa el intercambio que ocurre en el código:

```
if lista[j] > lista[j + 1]:  
    lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

Las variables j y $j+1$ son las responsables de ese movimiento.

- **Aporte de la Prueba de Escritorio:**

Gracias al rastreo manual comprendí mejor la función de los índices i y j .

i controla cuántos elementos finales ya están en su posición correcta.

j realiza las comparaciones dentro de cada pasada.

La prueba de escritorio permitió ver claramente por qué cada iteración reduce el número de comparaciones y cómo el algoritmo va “acercándose” al orden final.